Ch. 5: Constraining Subcategory-sensitive MATCH Constraints

Nicholas Van Handel (nvanhand@ucsc.edu), Dan Brodkin, Ben Eischens*

1 Introduction

Recent work at the syntax-prosody interface has argued that prosodic structure is built from a restricted inventory of recursive categories (Ito and Mester 2007, 2009, 2012, 2013; Selkirk 2011, inter alia). In the Prosodic Adjunction Theory of Ito and Mester (2007), for instance, recursive prosodic constituents are taken to be organized into subcategories based on their position in the prosodic tree, as shown in (1). For instance, φ are maximal when they are not dominated by any other φ and non-maximal when they are dominated by another φ. Similarly, φ are non-minimal when they dominate another φ and minimal when they do not. These subcategories can serve as the domains for different phonological processes, allowing for the identification of distinct prosodic domains without expanding the inventory of categories above the level of the foot beyond the phonological word (ω), phonological phrase (φ), and intonational phrase (ι) (Ito and Mester 2012, 2013; Elfner 2015; Elordieta 2015).

(1) Recursive φs are organized into prosodic subcategories (from Ito and Mester 2013, (5))

$$
\begin{array}{rcl}
 & \iota & \\
 & | & \\
\varphi^{[-\min]}\text{, the non-minimal projections} \left\{ \begin{array}{c} \\ \\ \\ \end{array} \right. & \varphi & \}\ \varphi^{[+\max,\,-\min]}\text{, the maximal projection} \\
 & | & \\
 & \varphi & \}\ \varphi^{[-\max,\,-\min]}\text{, an intermediate projection} \\
\varphi^{[-\max]}\text{, the non-maximal projections} \left\{ \begin{array}{c} \\ \\ \\ \end{array} \right. & | & \\
 & \varphi & \}\ \varphi^{[-\max,\,+\min]}\text{, the minimal projection} \\
 & | & \\
 & \omega &
\end{array}
$$

This understanding of prosodic structure forms the theoretical basis for the simplification of the process which relates syntactic structure to prosodic form in Selkirk's (2009, 2011) Match Theory. This theory holds that the mapping between syntactic and prosodic constituents is mediated by a set of Optimality-Theoretic constraints which enforce isomorphism between syntactic structures, organized into heads (X^0), phrases (XP), and clauses (CP), and prosodic structures, organized into ωs, φs, and ιs. This framework employs two types of these constraints. The first type governs the mapping from syntax to prosody and ensures that X^0s, XPs, and CPs are mapped to ωs, φs, and ιs. The second type governs the correspondence of prosody to syntax

* We thank Jenny Bellik and Junko Ito for helpful comments and suggestions on an earlier version of this chapter, and Eric Baković, Ryan Bennett, Richard Bibbs, Nick Kalivoda, Armin Mester, and Amanda Rysling for useful discussion of this work.

and ensures that ωs, φs, and ιs correspond to X^0s, XPs, and CPs, respectively. We provide examples of MATCH constraints at the phrasal level, along with their informal definitions, in (2).

(2) MATCH constraints at the phrasal level

a. sp.MATCH(XP, φ) (syntax-to-prosody)
“Ensure that each XP in the syntax corresponds to a φ in the prosody.”

b. ps.MATCH(φ, XP) (prosody-to-syntax)
“Ensure that each φ in the prosody corresponds to an XP in the syntax.”

Recent work has extended prosodic subcategories to the mapping constraints themselves, resulting in MATCH constraints that target specific subcategories (Ishihara 2014; Ito and Mester 2013, 2020; Kalivoda 2018). In his analysis of Japanese, for instance, Ishihara (2014) introduced the constraint sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$), which is defined in (3) below.

(3) sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$):
Assign one violation for each [+max] XP in the syntactic input that does not have a matching [+max] φ in the prosodic output.

This constraint differs from sp.MATCH(XP, φ) in two ways. Like all MATCH constraints, it contains a first argument (XP)[1] which delimits the set of constituents that need to be matched. In this case, the argument targets the set of maximal XPs rather than all XPs. The constraint also contains a second argument which expresses a requirement which holds over the correspondent. This second argument also makes reference to subcategories: the correspondent must be a maximal φ. In short, this constraint targets only a subset of the XPs targeted by general sp.MATCH(XP, φ) and places an additional requirement on the corresponding φs.

Ishihara introduced sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$) in order to derive the mapping from the left-branching input in (4) to the balanced structure in (4a). The perfectly matched candidate, (4c), is ruled out by BINMAX(φ, ω), which requires φ to contain at most two words. However, BINMAX(φ, ω) favors the flat candidate (4b) over the desired winner (4a), in which the φ corresponding to ZP contains four ωs. Ishihara concludes that matching ZP, which is maximal, is more important than respecting binarity. Ranking sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$) above BINMAX(φ, ω) ensures that the φ corresponding to ZP is preserved, while still allowing the ranking of BINMAX(φ, ω) over sp.MATCH(XP, φ) to rule out the perfectly isomorphic candidate.

[1] In the prosody-to-syntax constraint ps.MATCH(φ, XP), the first argument is φ instead of XP, but it has the same role of identifying those constituents that need a correspondent. Similarly, the second argument in this constraint is XP instead of φ, but it has the same role of stating what the correspondent needs to be.

(4) The constraint sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$) prioritizes matching ZP

$[_{ZP}\ [_{YP}\ [_{WP}\ [_{UP}$ u] w] y] z]	sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$)	BINMAX(φ, ω)	sp.MATCH(XP, φ)
→ a. $(_1\ (_2$ u w) $(_3$ y z))	0	1 φ_1	2 YP, UP
b. $(_1$ u w) $(_2$ y z)	W_1 ZP	L_0	W_3 ZP, YP, UP
c. $(_1\ (_2\ (_3\ (_4$ u) w) y) z)	e_0	W_2 φ_1, φ_2	L_0

This example demonstrates a key feature of subcategory-sensitive MATCH constraints: they target only a subset of XPs, which is useful when a prosodic markedness constraint prevents perfect matching. Other analyses have adopted Ishihara's constraint (Kalivoda 2018, Ito and Mester 2020, Bellik et al. to appear),[2] and Ito and Mester (2013) have proposed sp.MATCH($XP^{[-min]}$, φ), which requires all non-minimal XPs to have a φ correspondent. In the same vein, subcategory-sensitive ALIGN constraints have also been proposed (Selkirk and Elordieta 2010, Martinez-Paricio and Kager 2015). These analyses show that there is interest in the literature in augmenting CON with subcategory-sensitive constraints. However, opening the door to subcategory sensitivity allows for the possibility of constraints that favor theoretically implausible mappings, such as matching non-maximal XPs to maximal φs. The aim of this chapter is to systematically investigate the consequences of including such constraints in CON.

In Section 2, we show that subcategory sensitivity leads to a proliferation of MATCH constraints, some of which cause deviations from the syntax (*Anti-Match* effects) rather than enforcing syntax-prosody isomorphism. To determine which feature specifications cause Anti-Match effects, we systematically investigated the behavior of SSCs (subcategory-sensitive constraints) using the Syntax-Prosody in Optimality Theory (SPOT) application, which automatically generates and assigns violations to candidates (Bellik et al. 2015-2021). The methods and results of this investigation are described in Sections 3 and 4. In Section 5, we propose a ban on feature specifications that cause Anti-Match effects, arguing that this follows from MATCH constraints being part of the theory of Faithfulness. Section 6 outlines various approaches to extending the proposal that Anti-Match constraints do not belong in the theory, highlighting their advantages and disadvantages. Section 7 concludes.

---

[2] Bellik et al. (to appear) build the requirement that all maximal XPs are matched into GEN, but this is equivalent to having sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$) undominated in CON.

2 The problem: Unintended consequences of subcategory-sensitive MATCH

While subcategory-sensitive MATCH constraints can be useful, this augmentation of CON raises theoretical issues that have so far gone unaddressed. The first issue involves a proliferation of MATCH constraints. Consider the constraint sp.MATCH(XP, φ). Once subcategory-sensitivity is allowed, both arguments, XP and φ, can be specified for [min] and [max] status. For each feature, there are three possible values: [+, −, unspecified]. Four features (two for each argument) with three possible values each leads to 3^4, or 81, possible syntax-to-prosody MATCH constraints. This number doubles to 162 once prosody-to-syntax constraints are included. Though these constraints are logically possible, a theory that admits this many MATCH constraints introduces redundancy, and only a small fraction of these constraints have been motivated in the literature.

This proliferation of MATCH constraints is not merely an issue of parsimony. Many of the logically possible constraints predicted by this theory favor implausible mappings that directly enforce non-isomorphism between syntactic and prosodic structures, e.g., sp.MATCH($XP^{[-min]}$, $\varphi^{[+min]}$). We call constraints of this type *Anti-Match* because they favor at least one non-isomorphic candidate to the perfectly matching one. The inclusion of Anti-Match constraints in the theory would undermine the fundamental claim that MATCH constraints enforce syntax-prosody correspondence, while mismatches are the consequence of high-ranking prosodic markedness constraints (e.g., BINARITY, STRONGSTART) overriding MATCH constraints. Anti-Match constraints are also problematic from the perspective that MATCH constraints belong to the theory of Faithfulness. We return to these objections in detail in Section 5.

Anti-Match constraints come in two types. *Flattening* constraints favor ignoring at least one XP in the syntax, resulting in a prosodic output with fewer levels of embedding than the syntactic input. For instance, the constraint sp.MATCH($XP^{[-min]}$, $\varphi^{[+min]}$), defined in (5), attempts to map non-minimal XPs to minimal φs. For the right-branching input in (6), this constraint favors a non-isomorphic mapping. The SSC assigns a violation to the isomorphic candidate (6a): though non-minimal WP is matched, its correspondent φ_1 is not minimal, and the constraint is violated. Moreover, the SSC is satisfied by the flattened candidate (6b): failure to match the minimal YP ensures that the non-minimal WP corresponds to a minimal φ. Thus, if sp.MATCH($XP^{[-min]}$, $\varphi^{[+min]}$) were ranked above general sp.MATCH(XP, φ), a non-isomorphic candidate would win. This is a clear Anti-Match effect.

(5) sp.MATCH($XP^{[-min]}$, $\varphi^{[+min]}$): Assign one violation for each [−min] XP in the syntactic input that does not have a matching [+min] φ in the prosodic output.

(6) The constraint sp.MATCH($XP^{[-min]}$, $\varphi^{[+min]}$) favors flattening over isomorphism.

[$_{WP}$ w [$_{YP}$ y z]]	sp.MATCH(XP, φ)	ps.MATCH (φ, XP)	sp.MATCH ($XP^{[-min]}$, $\varphi^{[+min]}$)
→ a. ($_1$ w ($_2$ y z)) Isomorphic	0	0	1 WP
b. ($_1$ w y z) Flattened	W_1 YP	e_0	L_0

Expanding constraints work in the opposite direction: they favor building additional φs that are unmotivated by the syntax, thereby creating outputs with *more* levels of embedding than the syntactic input. Consider the constraint sp.MATCH($XP^{[+min]}$, $\varphi^{[-min]}$), defined in (7). As shown in (8), this SSC favors mapping the same right-branching input to outputs like (8b) and (8c) in which either *y* or *z* is parsed into a φ, even though neither *y* nor *z* constitutes an XP. Building this unmotivated φ structure ensures that the φ corresponding to YP is non-minimal, satisfying the SSC. In contrast, the isomorphic candidate is penalized because the minimal YP is matched to a minimal φ. This is another Anti-Match effect: (8b) and (8c) would win if sp.MATCH($XP^{[+min]}$, $\varphi^{[-min]}$) were ranked above ps.MATCH(φ, XP).

(7) sp.MATCH($XP^{[+min]}$, $\varphi^{[-min]}$): Assign one violation for each [+min] XP in the syntactic input that does not have a matching [−min] φ in the prosodic output.

(8) The constraint sp.MATCH($XP^{[+min]}$, $\varphi^{[-min]}$) favors expanding over isomorphism.

[$_{WP}$ w [$_{YP}$ y z]]	sp.MATCH(XP, φ)	ps.MATCH(φ, XP)	sp.MATCH($XP^{[+min]}$, $\varphi^{[-min]}$)
→ a. ($_1$ w ($_2$ y z)) Isomorphic	0	0	1 YP
b. ($_1$ w ($_2$ ($_3$ y) z)) Expanded	e_0	W_1 φ_3	L_0
c. ($_1$ w ($_2$ y ($_3$ z))) Expanded	e_0	W_1 φ_3	L_0

Though Flattening and Expanding constraints are clearly antithetical to the idea that MATCH constraints enforce isomorphism, it is less clear whether Anti-Match behavior is widespread, and determining the extent of the problem is non-trivial given that there exist many possible SSCs.

For this reason, we carried out an investigation in SPOT aimed at uncovering when Anti-Match effects arise.

3 SPOT Investigation

The Syntax-Prosody in Optimality Theory application is ideal for systematically studying SSCs because it allows for automatic generation and evaluation of candidates (Bellik et al. 2015-2021), which can then be used in OTWorkplace to calculate factorial typologies (Prince, Merchant, and Tesar 2007−2020). As discussed in Section 2, there are at least 81 logically possible syntax-to-prosody MATCH constraints. Given that each SSC must be tested across various syntactic inputs with a large set of possible prosodic outputs, investigating these constraints by hand would not have been feasible.

The goal of the investigation was to uncover which SSCs favor non-isomorphic candidates (*Anti-Match* constraints) and which ones favor the perfectly matched candidate (*Lawful Match* constraints). For each SSC under investigation, we developed an OT system. We kept GEN (i.e., the syntactic inputs and prosodic outputs) constant across all systems, while CON always contained sp.MATCH(XP, φ), ps.MATCH(φ, XP), and one SSC. For each system, we calculated a factorial typology.

If the factorial typology contained a single language, then the SSC was a Lawful Match constraint. This is because Lawful Match constraints never favor a non-isomorphic parse over the perfectly matched one. When CON contains sp.MATCH(XP,φ), ps.MATCH(φ, XP), and a Lawful Match SSC, the isomorphic candidate will emerge as the winner under any constraint ranking, resulting in one language.

If the factorial typology contained two languages, then the SSC was an Anti-Match constraint. To be Anti-Match, there must exist at least one syntactic input for which the constraint favors a non-isomorphic candidate over the isomorphic one. The constraint will therefore conflict with either sp.MATCH(XP, φ), if it prefers a flattened parse, or with ps.MATCH(φ, XP), if it prefers an expanded one. When CON contains sp.MATCH(XP,φ), ps.MATCH(φ, XP), and an Anti-Match constraint, different candidates will win depending on the ranking, resulting in a factorial typology with two languages.

In the results, we report whether each SSC was Lawful Match or Anti-Match, and if each Anti-Match constraint was a Flattening constraint or an Expanding constraint. With the logic of the study in place, we now define GEN and CON.

3.1 GEN

3.1.1 Inputs

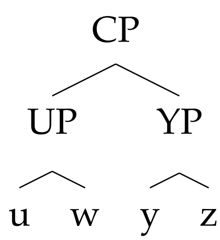
Syntactic inputs were generated using SPOT's automatic tree generator. The set of input trees consisted of all logically possible recursive nestings of 2–5 word strings with the properties in (9), such that all possible [max] and [min] specifications for XPs were included in the inputs. The complete list of all 24 inputs is provided in the Appendix.

(9) Inputs: all syntactic trees with the following properties:

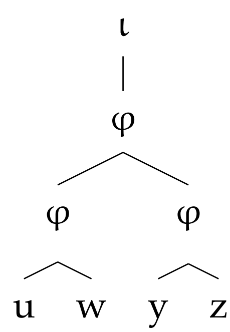
a. The root node is a CP.
b. All intermediate nodes are XPs.
c. All terminal nodes are X^0s.
d. There are between 2-5 terminal nodes.
e. All XP nodes are binary branching.
f. Heads can be anywhere.
g. Mirror image trees are excluded (e.g., the input {a [b c]} is included, but {[a b] c} is not, because these are mirror images).[3]

Trees were rooted in CP in order to include inputs like (10a), in which the maximal syntactic phrases, UP and YP, do not include the entire string of terminals.[4] This allows for the possibility of mapping a maximal XP to a non-maximal φ by building an even larger φ, as in (10b).

(10) a.



b.



For a similar reason, XP nodes are required to be binary branching so that all minimal XPs include more than one terminal node, as in (11a).[5] This allows for the possibility of mapping a minimal XP to a non-minimal φ by building an additional internal φ, as in (11b). If the minimal XP instead consisted of a single terminal node, it would be impossible to map the minimal XP to a non-minimal φ, because a single terminal node is already the smallest possible grouping. Without these inputs, we would miss potential Anti-Match effects due to an insufficiently rich GEN, threatening the validity of our conclusions about which constraints cause Anti-Match effects.

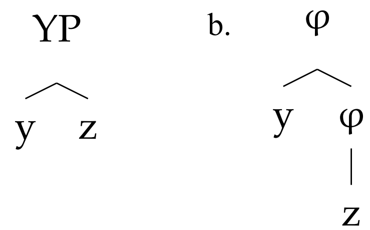
---

[3] Mirror image inputs are excluded because MATCH constraints are symmetrical, privileging neither the left nor right edge, and will assign the same number of violations to mirror image candidates. Omitting mirror images reduces the number of inputs without affecting the conclusions of the analysis.

[4] Following the distinction drawn between CP and other XPs in Match Theory, we assume that a maximal syntactic phrase is an XP (not CP) that is not dominated by any other XP. See also Ishihara (2014), who defines sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$) to see the maximal *lexical* phrase.

[5] In both trees and tableaux, CP and ι are shown only when a word in the syntactic input attaches directly to CP, without an intervening XP, as in (10). CP and ι are otherwise omitted, as in (11), to reduce visual clutter, but are assumed to be present in all structures.

(11) a. b.



3.1.2 Outputs

For each syntactic input, prosodic output candidates were generated with the properties in (12).

(12) Outputs: all prosodic trees with the following properties:

a. The root node is ι.
b. The intermediate nodes are φs.
c. The terminal nodes are ωs.
d. Every X^0 terminal node in the syntactic input is mapped to an ω terminal node in the prosodic output.
e. ι can immediately dominate ω (i.e., non-exhaustive parsing is allowed).
f. φ can dominate other φs (i.e., φ-recursion is allowed).
g. No φ dominates a φ with the same terminal nodes (i.e., vacuous recursion is prohibited).
h. The linear order of terminal nodes in the prosodic output is identical to the order in the syntactic input (i.e., movement is prohibited).

These settings allow for prosodic recursion and non-exhaustive parsing, resulting in a large number of candidates. This corresponds to the completely unrestricted core GEN in SPOT; see Shingler and Bellik (this volume, Ch. 2) for an explanation of the mathematics behind the number of candidates for an input with *n* terminal nodes, summarized in (13) for *n*=2 through *n*=5. Once 5-word inputs are considered, there are 2880 possible outputs given these GEN settings, underscoring the need for automation of candidate generation and constraint evaluation.

(13) Number of candidates for a tree with *n* terminal nodes.

Number of terminal nodes	Number of candidates
2	8
3	48
4	352
5	2880

3.2 CON

Each system's CON contained three constraints: sp.MATCH(XP, φ), ps.MATCH(φ, XP), and one of the SSCs. Given the large number of SSCs, we restricted ourselves to constraints that included either [max] or [min] specifications, but not both.[6] In the presentation of the results, we divide these constraints into four groups based on their feature specifications: (i) *Conflicting*: constraints in which the first and second arguments have opposite feature values, e.g., sp.MATCH($XP^{[+max]}$, $φ^{[-max]}$), (ii) *Second Argument Only*: constraints with feature specifications only on their second argument, e.g., sp.MATCH(XP, $φ^{[-max]}$), (iii) *First Argument Only*: constraints with specifications only on their first argument, e.g., sp.MATCH($XP^{[+max]}$, φ), and (iv) *Identical*: constraints in which the two arguments have the same feature values, e.g., sp.MATCH($XP^{[+max]}$, $φ^{[+max]}$). Each of these categories had four possible combinations of features, resulting in 4*4 = 16 syntax-to-prosody constraints.

4 Classifying subcategory-sensitive MATCH constraints

In this section, we present the results of the SPOT investigation for the 16 syntax-to-prosody constraints, classifying each constraint as Anti-Match or Lawful Match. We make additional distinctions within each class. As discussed above, there are two types of Anti-Match constraints: Flattening and Expanding. There are also three types of Lawful Match constraints. The *Specialized* constraints only care about matching a subset of XPs and are in a special-general relationship with sp.MATCH(XP, φ) because they always assign a subset of the violations assigned by the general constraint sp.MATCH(XP, φ). The *Superset* constraints are also in a special-general relationship with sp.MATCH(XP, φ), but assign a *superset* of the violations assigned by sp.MATCH(XP, φ); that is, the SSC is the general constraint. The *Dominance-Preserving* constraints are usually not in a special-general relationship with sp.MATCH(XP, φ) because they also prioritize the preservation of dominance relations by requiring XPs and φs to be in the same relative position in the tree.

---

[6] That is, constraints like sp.MATCH($XP^{[+max, -min]}$, φ) and sp.MATCH($XP^{[+max]}$, $φ^{[+min]}$), which involve both [min] and [max] features, were not considered.

We find that Anti-Match effects arise in two configurations: Conflicting and Second Argument Only specifications. In contrast, Lawful Match effects arise with Identical and First Argument Only specifications. We show that these patterns are related to the different roles of each argument: the first delimits the set of XPs that needs to be matched, and the second places requirements on the corresponding φ. After reviewing the generalizations from the SPOT investigation, we motivate principled limits on the use of SSCs in Section 5.

4.1 Conflicting specifications

We first consider those constraints for which the first and second arguments have conflicting feature specifications. We have already seen two of these constraints in Section 2: sp.MATCH($XP^{[-min]}$, $\varphi^{[+min]}$), a Flattening constraint, and sp.MATCH($XP^{[+min]}$, $\varphi^{[-min]}$), an Expanding constraint. Our investigation found that the other two syntax-to-prosody constraints with conflicting feature specifications are also Anti-Match: sp.MATCH($XP^{[+max]}$, $\varphi^{[-max]}$) is an Expanding constraint, while sp.MATCH($XP^{[-max]}$, $\varphi^{[+max]}$) is a Flattening constraint. The findings for this group are summarized in (14).

(14) Constraints with Conflicting specifications cause Anti-Match effects.

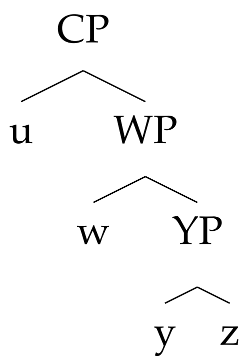
Constraint Name	Effect
sp.MATCH($XP^{[+max]}$, $\varphi^{[-max]}$)	Anti-Match: Expanding
sp.MATCH($XP^{[-max]}$, $\varphi^{[+max]}$)	Anti-Match: Flattening
sp.MATCH($XP^{[+min]}$, $\varphi^{[-min]}$)	Anti-Match: Expanding
sp.MATCH($XP^{[-min]}$, $\varphi^{[+min]}$)	Anti-Match: Flattening

Conflicting feature specifications are an explicit call for a change in dominance relations, such that an XP's relative position in the syntactic input differs from the relative position of the correspondent φ in the prosodic output. It is therefore unsurprising that these constraints cause Anti-Match effects. However, the Anti-Match problem is not limited to SSCs with opposite feature values: in the next section we show that Anti-Match effects arise even when feature specifications are limited to the second argument of the MATCH constraint.

4.2 Specifications on the Second Argument Only

The Anti-Match problem generalizes to constraints that only have feature specifications on the second argument. Consider how the constraint sp.MATCH(XP, $\varphi^{[-max]}$) treats potential prosodic parses of the right-branching input in (15). The first argument of the constraint lacks any feature specification and therefore picks out all XPs: maximal WP and non-maximal YP. The second argument states that a violation is incurred for each one of these XPs that fails to map to a non-maximal φ. Because the maximal WP is included in the set of all XPs, this constraint still calls for a change in dominance relations; this call is simply implicit, hidden by the lack of any features on the constraint's first argument.

(15)



The tableau in (16) shows that this is an Expanding constraint, as expected given the implicit call for a change in dominance relations. The isomorphic candidate (16a) violates sp.MATCH(XP, $\varphi^{[-max]}$) once because WP is not matched to a non-maximal φ. In contrast, the expanded candidate (16b) avoids this violation by building an additional φ above the one corresponding to WP, at the cost of incurring a violation of ps.MATCH(φ,XP). As with other Expanding constraints, sp.MATCH(XP, $\varphi^{[-max]}$) will choose the non-isomorphic candidate (16b) when ranked above ps.MATCH(φ,XP).

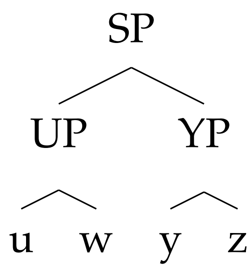
(16) sp.MATCH(XP, $\varphi^{[-max]}$) favors the expanded candidate.

{$_{CP}$ u [$_{WP}$ w [$_{YP}$ y z]]}	sp.MATCH(XP, φ)	ps.MATCH(φ, XP)	sp.MATCH(XP, $\varphi^{[-max]}$)
→ a. {u ($_1$ w ($_2$ y z))} Isomorphic	0	0	1 WP
b. {$_\iota$ ($_1$ u ($_2$ w ($_3$ y z)))} Expanded	e_0	W_1 φ_1	L_0

The constraint sp.MATCH(XP, $\varphi^{[-min]}$) is an Expanding constraint for a similar reason: the first argument, XP, picks out all XPs in the syntactic tree, and this set will always include at least one minimal XP. This SSC will favor expanded candidates in which the minimal XP is mapped to a non-minimal φ by building an additional φ below the one corresponding to the minimal XP.

Specifications on the second argument can also cause Flattening. For the input in (17), the SSC sp.MATCH(XP, $\varphi^{[+max]}$) once again has an implicit call for a change in dominance relations: the set of XPs delimited by the first argument includes non-maximal UP and non-maximal YP, which the constraint favors mapping to maximal φ.

(17)



Again, the pressure to change the [max] feature in the prosody leads to Anti-Match effects: the isomorphic candidate (18a) violates the SSC twice because it maps neither UP nor YP to maximal φ, while the non-isomorphic candidate (18b) only violates the SSC once by ignoring the top layer SP. When the SSC is ranked above sp.MATCH(XP, φ), the flattened candidate will win.

(18) sp.MATCH(XP, $\varphi^{[+max]}$) favors the flattened candidate.

[$_{SP}$ [$_{UP}$ u w] [$_{YP}$ y z]]	sp.MATCH(XP, φ)	ps.MATCH(φ, XP)	sp.MATCH(XP, $\varphi^{[+max]}$)
→ a. ($_1$ ($_2$ u w) ($_3$ y z)) Isomorphic	0	0	2 UP, YP
b. ($_1$ u w) ($_2$ y z) Flattened	W_1 SP	e_0	L_1 SP

The last constraint of this category, sp.MATCH(XP, $\varphi^{[+min]}$), never causes Anti-Match effects. Given what we have seen, this is an unexpected result: whenever the input contains non-minimal XPs, the constraint introduces a pressure to map these non-minimal XPs to minimal φs. This is another implicit call for a change in dominance relations, so why do we not find Anti-Match effects?

The answer lies in tree geometry. First, consider how sp.MATCH(XP, $\varphi^{[+min]}$) assigns violations. An isomorphic parse will map all and only the non-minimal XPs in the input to non-minimal φ, thereby incurring a violation for each non-minimal XP. A flattened candidate will try to perform better by matching non-minimal XPs to minimal φs. However, non-minimal XPs always contain one or more XPs by definition. Any benefit from matching a non-minimal XP to a minimal φ will be offset by the failure to match the one or more XPs that it dominates, so flattening can never improve on the isomorphic parse.

To see how this works concretely, the tableau in (19) shows how sp.MATCH(XP, $\varphi^{[+min]}$) prefers to parse the structure from (17). The isomorphic parse (19a) incurs a single violation of the SSC for failing to map non-minimal SP to a minimal φ. The flattened (19b) successfully maps SP to a minimal φ, but ends up performing worse because it fails to match both UP and YP. Thus, the isomorphic candidate wins despite the constraint's implicit call for a change in dominance relations.

(19) sp.MATCH(XP, $\varphi^{[+min]}$) does not favor the flattened parse, even when the input contains non-minimal XPs whose feature value could be reversed.

$[_{SP}\ [_{UP}$ u w] $[_{YP}$ y z]]	sp.MATCH(XP, φ)	ps.MATCH(φ, XP)	sp.MATCH(XP, $\varphi^{[+min]}$)
→ a. $(_1\ (_2$ u w) $(_3$ y z)) Isomorphic	0	0	1 SP
HB b. $(_1$ u w y z) Flattened	W_2 UP, YP	e_0	W_2 UP, YP

Before moving on, we point out that sp.MATCH(XP, $\varphi^{[+min]}$) is also in a special-general relationship with sp.MATCH(XP, φ). This stringency relationship arises when the violations assigned by the special constraint are a subset of those assigned by the general one. In this case, sp.MATCH(XP, φ) is the special constraint, because its violations are a subset of those assigned by the SSC. In both constraints, the first argument dictates that all XPs need to be mapped to some φ; thus, whenever an XP is not matched in the output, both constraints are violated. However, the constraints differ in how they assign violations when XPs *are* matched in the output. The constraint sp.MATCH(XP, φ) is satisfied as long as there is some φ, regardless of its position in the tree, while the second argument of the SSC requires that the matching φ be minimal. XPs mapped to non-minimal φ will satisfy sp.MATCH(XP, φ) but not the SSC, explaining why the latter's violations are a superset of those assigned by sp.MATCH(XP, φ).[7]

Intuitively, it may seem strange that sp.MATCH(XP, φ) is the special constraint: we have informally referred to sp.MATCH(XP, φ) as a general constraint, and sp.MATCH(XP, $\varphi^{[+min]}$) may seem "special" since its second argument bears an additional feature specification. Nevertheless, the terminology reflects the formal definition based on the violations assigned by each constraint. We call sp.MATCH(XP, $\varphi^{[+min]}$) a *Superset* constraint, to differentiate it from the SSCs in Section 4.3, which assign a subset of the violations assigned by sp.MATCH(XP, φ).

Zooming out, we find that Anti-Match effects still arise when only the second argument of a SSC has feature specifications, with one exception due to tree geometry; these findings are summarized in the table in (20). We showed that Anti-Match effects arise despite the absence of an explicit feature conflict between the two arguments in the constraint definitions, because the set of all XPs may still contain an XP with a feature value opposite of the one specified on the second argument. These findings suggest that Anti-Match effects are not always immediately apparent from a constraint definition. They also suggest that SSCs that have feature specifications on the second argument that are not also present on the first argument should be avoided; we take this up in more detail in Section 5.

[7] In fact, the three other Second Argument Only constraints investigated here also assign a superset of the violations assigned by sp.MATCH(XP, φ), for similar reasons. As described earlier, they differ from sp.MATCH(XP, $\varphi^{[+min]}$) in that they also cause Anti-Match effects.

(20) Constraints with Second Argument Only specifications often, but not always, cause Anti-Match effects.

Constraint Name	Behavior
sp.MATCH(XP, $\varphi^{[-max]}$)	Anti-Match: Expanding
sp.MATCH(XP, $\varphi^{[+max]}$)	Anti-Match: Flattening
sp.MATCH(XP, $\varphi^{[-min]}$)	Anti-Match: Expanding
sp.MATCH(XP, $\varphi^{[+min]}$)	Lawful Match: Superset

4.3 Specifications on the First Argument Only

In contrast to the previous group of constraints, no Anti-Match effects are found when the feature specification is only on the first argument. Instead, these SSCs assign a subset of the violations assigned by sp.MATCH(XP, φ), with which they are in a special-general relationship. This is because the first argument of a SSC like sp.MATCH($XP^{[+min]}$, φ) picks out a subset of XPs to match, while the general sp.MATCH(XP, φ) cares about matching all XPs. We therefore call SSCs with First Argument Only specifications *Specialized* constraints.

The tableau in (21) illustrates the special-general relationship between sp.MATCH(XP, φ) and sp.MATCH($XP^{[+min]}$, φ) with a right-branching input. The isomorphic (21a) satisfies both constraints by matching everything. The partially flattened (21b), which fails to match YP, incurs one violation of both sp.MATCH(XP, φ) and the SSC. The completely flattened (21c) incurs two violations of sp.MATCH(XP, φ) by failing to match WP and YP but only one violation of the SSC. Across all three candidates, we see that sp.MATCH($XP^{[+min]}$, φ) assigns only a subset of the violations assigned by sp.MATCH(XP, φ), as expected.

(21) sp.MATCH($XP^{[-min]}$, φ) and sp.MATCH(XP, φ) are in a special-general relationship.

[$_{UP}$ u [$_{WP}$ w [$_{YP}$ y z]]]	sp.MATCH(XP, φ)	ps.MATCH(φ, XP)	sp.MATCH($XP^{[+min]}$, φ)
→ a. ($_1$ u ($_2$ w ($_3$ y z))) Isomorphic	0	0	0
HB b. ($_1$ u ($_2$ w y z)) Flattened	W_1 YP	e_0	W_1 YP
HB c. ($_1$ u w y z) Flattened	W_2 WP, YP	e_0	W_1 YP

Although we only demonstrate the special-general relationship for sp.MATCH($XP^{[+min]}$, φ), we found that the same relationship holds between sp.MATCH(XP, φ) and the other three SSCs with specifications on the first argument, as summarized in (22). The lack of Anti-Match effects due to the special-general relationship suggests that constraints with specifications only on the first argument may not be a problematic addition to the theory of Match constraints.

(22) Constraints with First Argument Only specifications are Specialized.

Constraint Name	Behavior
sp.MATCH(XP$^{[+max]}$, φ)	Lawful Match: Specialized
sp.MATCH(XP$^{[−max]}$, φ)	Lawful Match: Specialized
sp.MATCH(XP$^{[+min]}$, φ)	Lawful Match: Specialized
sp.MATCH(XP$^{[−min]}$, φ)	Lawful Match: Specialized

4.4 Identical specifications

SSCs in which the arguments have identical feature specifications are also Lawful Match constraints, but the majority of them are not in a special-general relationship with sp.MATCH(XP, φ). Consider the constraint sp.MATCH(XP$^{[+min]}$, φ$^{[+min]}$). Like the SSCs in Section 3.3, this SSC only attempts to match a subset of those XPs targeted by sp.MATCH(XP, φ) due to the [+min] feature on the first argument. However, this SSC sometimes assigns more violations than sp.MATCH(XP, φ) because it also requires the XP and φ to share certain features, whereas sp.MATCH(XP, φ) is satisfied as long as some matching φ exists. We call this type of constraint Dominance-Preserving, because requiring XP and φ to have identical feature specifications ensures that their relative position is the same in both syntax and prosody.

This dominance preservation is shown in (23) for a four-word right-branching syntactic input. The fully flattened (23b) incurs violations of both sp.MATCH(XP, φ) and the SSC by failing to match certain XPs. The expanded constraint (23c) satisfies sp.MATCH(XP, φ), because all XPs are matched to some φ. However, it violates the SSC: although YP is matched, its corresponding φ is minimal, so its relative position in the tree structure has changed.

(23) sp.MATCH($XP^{[+min]}$, $\varphi^{[+min]}$) and sp.MATCH(XP, φ) are not in a special-general relationship.

$[_{UP}$ u $[_{WP}$ w $[_{YP}$ y z]]]	sp.M(XP, φ)	ps.M(φ, XP)	sp.M($XP^{[+min]}$, $\varphi^{[+min]}$)
→ a. $(_1$ u $(_2$ w $(_3$ y z))) Isomorphic	0	0	0
HB b. $(_1$ u w y z) Flattened	W_2 WP, YP	e_0	W_1 YP
HB c. $(_1$ u $(_2$ w $(_3$ y $(_4$ z)))) Expanded	e_0	W_1 φ_4	W_1 YP

Thus, Dominance-Preserving constraints sometimes assign more violations than sp.MATCH(XP, φ) because they place additional requirements on matching φs. In addition to sp.MATCH($XP^{[+min]}$, $\varphi^{[+min]}$), we found that sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$), sp.MATCH($XP^{[-max]}$, $\varphi^{[-max]}$), and sp.MATCH($XP^{[-min]}$, $\varphi^{[-min]}$) were also Dominance-Preserving constraints.

Interestingly, sp.MATCH($XP^{[-min]}$, $\varphi^{[-min]}$) turned out to be in a special-general relationship with sp.MATCH(XP, φ), unlike the other Dominance-Preserving constraints. That is, the number of violations assigned by sp.MATCH($XP^{[-min]}$, $\varphi^{[-min]}$) is always a subset of those assigned by sp.MATCH(XP, φ). Why should this be the case? Consider the two ways to violate sp.MATCH($XP^{[-min]}$, $\varphi^{[-min]}$). First, a non-minimal XP could fail to be mapped to *any* φ; this will violate both the SSC and general sp.MATCH(XP, φ), so does not affect the special-general relation. Alternatively, a non-minimal XP could be mapped to a minimal φ; this minimal φ would violate the SSC but would satisfy sp.MATCH(XP, φ). However, this mapping will be accompanied by one or more violations of sp.MATCH(XP, φ), because it requires ignoring all XPs inside the non-minimal XP, which contains at least one XP by definition. Thus, the total violations assigned by sp.MATCH(XP, φ) will always be greater than or equal to those assigned by the SSC, resulting in the special-general relationship.

This type of mapping is illustrated in (24): the non-minimal SP in (24a) is mapped to a minimal φ in (24b) by ignoring UP and YP. This leads to one violation of sp.MATCH($XP^{[-min]}$, $\varphi^{[-min]}$) due to SP and two violations of sp.MATCH(XP, φ) due to UP and YP. This demonstrates how a violation of sp.MATCH($XP^{[-min]}$, $\varphi^{[-min]}$) caused by mapping a non-minimal XP to a minimal φ will always co-occur with one or more violations of sp.MATCH(XP, φ). It should also be noted that sp.MATCH($XP^{[-min]}$, $\varphi^{[-min]}$) is still a Dominance-Preserving constraint: it is only satisfied when non-minimal SP is mapped to a non-minimal φ. In contrast, the Specialized constraint sp.MATCH($XP^{[-min]}$, φ) would be satisfied by the Flattened (24b) because it does not care whether SP retains its non-minimal status as long as it is matched.

(24) sp.MATCH($XP^{[-min]}$, $\varphi^{[-min]}$) is both Dominance-Preserving and in a special-general relationship with sp.MATCH(XP, φ)

[SP [UP u w] [YP y z]]			sp.M(XP, φ)	ps.M(φ, XP)	sp.M($XP^{[-min]}$, $\varphi^{[-min]}$)
→	a.	($_1$ ($_2$ u w) ($_3$ y z))) Isomorphic	0	0	0
HB	b.	($_1$ u w y z) Flattened	W_2 UP, YP	e_0	W_1 SP

As summarized in (25), identical feature specifications lead to a second type of Lawful Match, the Dominance-Preserving constraints, which aim not only to match constituents but to preserve the relative position of nodes in the tree. Unlike Specialized constraints, Dominance-Preserving constraints are not necessarily in a special-general relationship with sp.MATCH(XP, φ). As with Specialized constraints, the absence of Anti-Match effects suggest that Dominance-Preserving constraints can be permitted in the theory without causing issues, and we briefly consider how Specialized and Dominance-Preserving constraints might be teased apart in Section 6.1.

(25) Constraints with Identical specifications are Dominance-Preserving.

Constraint Name	Behavior
sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$)	Lawful Match: Dominance-Preserving
sp.MATCH($XP^{[-max]}$, $\varphi^{[-max]}$)	Lawful Match: Dominance-Preserving
sp.MATCH($XP^{[+min]}$, $\varphi^{[+min]}$)	Lawful Match: Dominance-Preserving
sp.MATCH($XP^{[-min]}$, $\varphi^{[-min]}$)	Lawful Match: Dominance-Preserving (special)

4.5 Interim summary

The SPOT investigation showed that Anti-Match effects are fairly widespread: we found multiple Flattening and Expanding constraints among the 16 SSCs investigated here. The summary in (26) shows the number of Anti-Match and Lawful Match constraints for each configuration; as a reminder, four constraints were tested in each configuration. This summary shows that Anti-Match effects are limited to two configurations: Conflicting and Second Argument Only feature specifications. In the following section, we argue that these configurations should be excluded from the theory in order to avoid Anti-Match effects.

(26) Number of Anti-Match and Lawful Match SSCs for each configuration.

Feature Specifications	Anti-Match	Lawful Match
Conflicting	4	0
Second Argument Only	3	1
First Argument Only	0	4
Identical	0	4

5 Constraining subcategory-sensitive constraints

The results of this investigation lead to an architectural conclusion about the licit shapes of MATCH constraints in the grammar. In the preceding sections, we have seen that the conceptual allowance for unconstrained feature specifications yields a drastic increase in the generative capacity of the system with little, if any, empirical gain. This observation suggests that the theory must be constrained in some way to rule out the presence of Anti-Match constraints.

To achieve this goal, we propose that Conflicting and Second Argument Only specifications are prohibited. These configurations cause the problematic Anti-Match effects because they require, either explicitly or implicitly, for XPs and their φ correspondents to have conflicting feature values. We refer to this proposal as the *Ban on Anti-Match Constraints*, stated in (27). As a consequence of this ban, MATCH constraints can only place a specification on their second argument if the same specification is also present on the first argument.[8]

(27) The Ban on Anti-Match Constraints: MATCH constraints must not…

a. Place conflicting subcategory specifications on their first and second arguments (Conflicting)

b. Place a feature specification on their second argument without a parallel specification on the first argument (Second Argument Only)

This proposal forms an integral component of the broader theory of MATCH constraints: it addresses the problem of overgeneration identified in Sections 3 and 4 and flows naturally from the fundamental logic of the theory of syntax-prosody mapping which has emerged from work in Prosodic Adjunction Theory (Ito and Mester 2007, 2009) and Match Theory (Selkirk 2011, Ishihara 2014, Elfner 2015, Elordieta 2015). In Section 6, we will discuss several ways in which this proposal might be extended. Before taking up this task, we lay out two arguments, both within the theoretical framework which we adopt, which we take to motivate its necessity.

[8] Note that the Ban on Anti-Match constraints also prohibits constraints like sp.MATCH($XP^{[+min]}$, $\varphi^{[-max]}$) and sp.MATCH($XP^{[+min]}$, $\varphi^{[-max, +min]}$). While these constraints have specifications on both arguments, these constraints fall under the umbrella of Second Argument Only specifications because the feature [−max] is specified only for the second argument. In contrast, the constraint sp.MATCH($XP^{[-max, +min]}$, $\varphi^{[-max]}$) would be allowed, because the [−max] specification on the second argument is also present on the first.

5.1 *The Nature of the Syntax-Prosody Mapping*

The principal consideration which militates against the Anti-Match constraints above is an observation about the nature of divergence between syntactic and prosodic structure. Since the earliest period of work at this interface, it has been recognized that prosodic organization mirrors the syntax which underlies it in a close but imperfect manner (Chomsky and Halle 1968, Clements 1978, Chen 1987, Selkirk 1986). In light of this fact, it has become a theoretical priority to understand the nature of such divergences between syntactic and prosodic form. The leading strand of thought in this literature, which assumes the existence of a distinct level of prosodic representation, has historically approached this type of discrepancy in one of two ways.

The earliest approach to this puzzle proposed to connect such syntax-prosody mismatches to properties of the mapping itself. The pioneering investigations of the interface in the 1980s, for instance, typically assumed a view of prosodic structure, now referred to as the Strict Layer Hypothesis (Nespor and Vogel 1986, Selkirk 1986), which held that prosodic organization differed from syntactic structure in fundamental respects (lacking, for instance, the capacity for recursion). As a natural consequence of this assumption, the literature of this era generally did not envision the existence of a system of mapping which produces prosodic structures with the same organization as the syntax which underlies them. Rather, these works often assumed a system of translation which rearranged syntactic structures into geometries that complied with constraints on prosodic organization. The end-based theory of Selkirk (1986), for instance, is one model of this type.

These approaches contrast with an alternative which assumes that mismatches between the syntax and prosody do not arise from constraints on mapping itself, but rather from independent pressures which govern prosodic form. This alternative, which maintains the central intuition of Match Theory, differs from its predecessors in several respects. First, it recognizes the observation—one which has now been defended at every level of the prosodic hierarchy above the foot[9]—that prosodic structure permits recursion (Ladd 1986; Booij 1996; Peperkamp 1997; Vigário 1999, 2003; Ito and Mester 2007, 2009, 2013; Wagner 2005; Elfner 2015; Bennett 2018) and other deviations from the structural ideal once assumed by proponents of Strict Layering. Second, it adopts the theoretical stance that prosodic structure above the foot is organized into a restricted set of recursive categories: the prosodic word, phonological phrase, and intonational phrase (Ito and Mester 2007, 2009; Elfner 2015; Elordieta 2015; Bennett 2018).

These two advances reflect the central theoretical insights of the papers which laid out the theory of prosodic structure which has come to be known as Prosodic Adjunction Theory (Ito and Mester 2007, 2009). Their adoption, in turn, paves the way for a drastic simplification of the process which maps syntactic structure into prosodic form, as developed in Match Theory (Selkirk 2011). Should prosodic structure support recursion and be organized into categories with direct analogues in the syntax, it becomes possible to understand the process of mapping as

[9] Prosodic recursion has also been argued for at the level of the foot (Bennett 2012, Martínez-Paricio and Kager 2015, Kager and Martínez-Paricio 2018, i.a.).

one which prefers an exact one-to-one translation of the constituent structure of a syntactic input into a prosodic output analogous in all respects. This is a view on which the mapping itself does not impose deviations between the syntactic input and the prosodic output, and any mismatches between syntax and prosody are instead linked to independent prosodic well-formedness constraints. [10]

Proponents of Match Theory have formalized the workings of this mapping in several different ways (Selkirk 2009, 2011; Elfner 2015; Ito and Mester 2019; Lee and Selkirk to appear). What is important at present, however, is that the intuition above reflects a central tenet of the theory. In our view, there are good reasons for this to be so: the notion that the mapping constraints do not drive mismatch is one which fits well with the observation that prosodic structure can be organized much like syntax and yields genuine theoretical simplification in terms of the mapping between the two.

In light of this stance, we arrive at a theoretical consideration which suggests the need to rule out Anti-Match constraints. This is the observation that Anti-Match constraints explicitly drive mismatches between the syntax and the prosody via Flattening and Expanding, contra the claim that such deviations are the responsibility of prosodic markedness constraints.

5.2 Match and the Theory of Faithfulness

The arguments above lead us to the view that Anti-Match constraints run against the spirit of Match Theory. As such, we submit that they have no place in the theory at all. Simple as this conclusion may seem, it yields immediate benefits on both the empirical and theoretical levels. First, it excludes the constraints which the SPOT investigation identified as motivating mismatches for non-phonological reasons in Section 4. Second, it leads to a substantial reduction in the number of MATCH constraints in the theory and delivers a more constrained typology of predicted languages. Third and finally, it allows MATCH constraints to be integrated into the theory of Faithfulness.

This final point, though theory-internal in nature, provides a strong secondary motivation for the complete exclusion of Anti-Match constraints from the theory. In essence, it turns on the formal implementation of Match Theory in Optimality Theory. Within this framework, the constraints which enforce isomorphism between syntax and prosody are typically taken to fall under the umbrella of *Faithfulness*: that is, they enforce a type of cross-modular correspondence between syntactic and prosodic constituents. In the formulation of these constraints, for instance, Selkirk (2009:17) writes,

---

[10] The one exception is that certain XPs, like functional phrases (FuncP), may be invisible to mapping constraints (Nespor and Vogel 1986, Selkirk 1986, Truckenbrodt 1999, Lee and Selkirk to appear, i.a.). While sp.MATCH(LexP, φ) introduces a pressure to match lexical XPs (LexP), it does not drive mismatches because it does not penalize φs that correspond to FuncP. Like Specialized SSCs, it only introduces the pressure to match a subset of XPs. However, ps.MATCH(φ, LexP) can drive mismatches, because it militates against all φs that lack a LexP correspondent, including φs that correspond to FuncP. This is similar to Flattening SSCs: ps.MATCH(φ, LexP) prefers a structure with fewer levels of embedding than if all XPs were matched. We view this as a principled exception, different from Anti-Match SSCs, because (i) lexical-only MATCH constraints still prefer perfect matching of LexP's, and (ii) there is ample evidence that a Lex/Func distinction (or a similar restriction) is needed (Van Handel this volume, Ch. 6).

"The first type—call them S-P faithfulness constraints—require that syntactic constituency be faithfully reflected in prosodic constituency, and the second—call them P-S faithfulness constraints—require that prosodic constituency be a faithful reflection of syntactic constituency."

The formalization of MATCH constraints as a part of the theory of Faithfulness leads to two implications about the properties of these constraints. The first is that the theory must allow these constraints to treat certain elements of syntactic and prosodic structure as "correspondent" in some way. At the lowest level, MATCH constraints treat the pairings of X^0:ω, XP:φ, and CP:ι to reflect fundamentally faithful mappings. In this sense, these constraints are held to enforce correspondence between categorial labels. At a higher level, it follows from a theory which assumes the possibility for exact mapping that these constraints may force the preservation of dominance relations as well: under ideal circumstances, they may require that the phrases which are undominated in the syntax ("syntactically maximal") be similarly undominated in the prosody ("prosodically maximal") and that the phrases dominated in the syntax ("syntactically non-maximal") be similarly dominated in the prosody ("prosodically non-maximal"). In this respect, we might assume that MATCH constraints bear responsibility for the correspondence of subcategory labels as well.

A second concern follows from the inclusion of MATCH constraints in the theory of Faithfulness. This lies in the types of requirements which can licitly be imposed on outputs by constraints of this type. In Optimality Theory, faithfulness constraints serve as the formal machinery which ensures minimal difference between input and output. As their role is to preserve properties of the input, it is a definitional property of such constraints that they cannot force outputs to differ from inputs in any way. This observation leads to a clear conclusion about the shapes of MATCH constraints: if they are understood as faithfulness constraints in any meaningful sense, they should be unable to force differences between their outputs and inputs.

With this conclusion in hand, we arrive at a second strike against the Anti-Match constraints above: they do not behave like faithfulness constraints, because they place requirements on outputs which cannot be connected to properties of their inputs. As such, they cannot be subsumed into the theory of Faithfulness with the Lawful Match constraints.

We take this observation to provide theoretical motivation for the elimination of the Anti-Match constraints above. The general profile of the mapping from syntax to prosody is one where prosodic structure appears faithful to syntactic constituency, and as such, the constraints which govern this mapping should be conceptualized as part of the theory of Faithfulness. However, the admission of Anti-Match constraints would render it impossible to categorically associate MATCH constraints with Faithfulness in this way. To allow them in at this cost would appear to be a mistake. As such, we suggest, they must be ruled out from the theory.

These two arguments lead us to the Ban on Anti-Match Constraints, stated earlier in (27), which prohibits Conflicting and Second Argument Only specifications. This proposal ensures that MATCH constraints do not impose subcategory specifications on their second argument that are not also present on their first argument. As a result, the sole MATCH constraints allowed in CON are those which bear the shapes in (28).[11]

(28) Permissible constraint shapes under the Ban on Anti-Match Constraints

a. Licit SP constraints:
   i. sp.MATCH(X^0, ω), sp.MATCH(XP, φ), sp.MATCH(CP, ι)
   ii. sp.MATCH($X^{0[+max]}$, ω), sp.MATCH($XP^{[+min]}$, φ), ...
   iii. sp.MATCH($X^{0[+max]}$, $\omega^{[+max]}$), sp.MATCH($XP^{[+min]}$, $\varphi^{[+min]}$), ...

b. Licit PS constraints:
   i. ps.MATCH(ω, X^0), ps.MATCH(φ, XP), ps.MATCH(ι, CP)
   ii. ps.MATCH($\omega^{[+max]}$, X^0), ps.MATCH($\varphi^{[+min]}$, X^0), ...
   iii. ps.MATCH($\omega^{[+max]}$, $X^{0[+max]}$), ps.MATCH($\varphi^{[+min]}$, $XP^{[+min]}$), ...

6 Extending the proposal

The discussion above has led us to the proposal that, on the grounds of general considerations on the nature of Faithfulness and Match Theory, the subcategory-sensitive MATCH constraints which lead to Anti-Match effects should be excluded from CON. As shown in the SPOT investigation, these constraints are those in which the second argument has a feature specification that is not also present on the first, e.g., sp.MATCH($XP^{[+max]}$, $\varphi^{[-max]}$) and sp.MATCH(XP, $\varphi^{[+max]}$). Eliminating these constraints narrows down the number of logically possible MATCH constraints, simplifying CON and the resulting typology. More importantly, it prevents Anti-Match effects, solving the main theoretical issue raised in this chapter.

However, one might extend the proposal outlined above in various ways, some of which are more restrictive than others. In this section, we outline possible extensions, as well as one alternative that lies outside the purview of this proposal but nonetheless merits discussion.

6.1 Specialized vs. Dominance-Preserving Constraints

The prohibition on Anti-Match constraints in Section 5 still allows for two distinct types of Lawful Match constraints: Specialized constraints, which arise with First Argument Only specifications (e.g., sp.MATCH($XP^{[+max]}$, φ), and Dominance-Preserving constraints, which arise with Identical specifications (e.g., sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$). However, one might further constrain the inventory of subcategory-sensitive MATCH constraints by only allowing one type and not the other. For example, one might suppose that either Specialized or Dominance-

[11] Though not discussed here for reasons of space, we found that prosody-to-syntax SSCs show similar behavior to the syntax-to-prosody SSCs: only Conflicting and Second Argument Only specifications cause Anti-Match effects. Again, this is because specifications on the second argument that are not also present on the first require correspondents to have different feature values. The Ban on Anti-Match constraints is therefore general, applying to both syntax-to-prosody and prosody-to-syntax constraints.

Preserving constraints are necessary, but not both. This point is especially apparent when one considers that Specialized and Dominance-Preserving Match constraints often do the same work. For example, Kalivoda (2018) and Ito and Mester (2020) use the Specialized constraint sp.MATCH($XP^{[+max]}$, φ) to account for the same phenomenon as the Dominance-Preserving sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$) adopted in Ishihara (2014).

There are two possible forms of this approach. The first, which we refer to as the First Argument Only Hypothesis, states that subcategory specifications are allowed in MATCH constraints, but only on the first argument. On such an approach, only the constraints with the shape in (29) would be allowed, while constraints with identical specifications would be prohibited.

(29) The First Argument Only Hypothesis

a. Licit SP constraints:
   i. sp.MATCH(X^0, ω), sp.MATCH(XP, φ), sp.MATCH(CP, ι)
   ii. sp.MATCH($X^{0[+max]}$, ω), sp.MATCH($XP^{[+min]}$, φ), …

b. Licit PS constraints:
   i. ps.MATCH(ω, X^0), ps.MATCH(φ, XP), ps.MATCH(ι, CP)
   ii. ps.MATCH($\omega^{[+max]}$, X^0), ps.MATCH($\varphi^{[+min]}$, X^0), ...

An alternative is the Identical Specifications Hypothesis, which states that constraints with identical specifications are the only permissible subcategory-sensitive MATCH constraints. Under this view, constraints of the shape in (30) would be permissible, and the First Argument Only constraints in (29) would be prohibited.

(30) The Identical Specifications Hypothesis

a. Licit SP constraints:
   i. sp.MATCH(X^0, ω), sp.MATCH(XP, φ), sp.MATCH(CP, ι)
   ii. sp.MATCH($X^{0[+max]}$, $\omega^{[+max]}$), sp.MATCH($XP^{[+min]}$, $\varphi^{[+min]}$), ...

b. Licit PS constraints:
   i. ps.MATCH(ω, X^0), ps.MATCH(φ, XP), ps.MATCH(ι, CP)
   ii. ps.MATCH($\omega^{[+max]}$, $X^{0[+max]}$), ps.MATCH($\varphi^{[+min]}$, $XP^{[+min]}$), ...

Though we do not take a position on these two approaches here, it is important to point out that whether the theory should include only Specialized or only Dominance-Preserving constraints is ultimately an empirical question, since each type of constraint favors slightly different prosodic structures for certain inputs. For example, imagine a hypothetical language in which the input structure in (31) is prosodified as shown.

(31) $[_{WP}$ w $[_{YP}$ y z]] → (w y z)

This mapping could be modeled by ranking NONRECURSIVITY over sp.MATCH(XP, φ), motivating flattening, as shown in (32).

(32) NONREC favors the flattened candidate.

[$_{WP}$ w [$_{YP}$ y z]]	NONREC	sp.MATCH(XP, φ)
→ a. ($_1$ w y z) Flattened	0	1 YP
b. ($_1$ w ($_2$ y z)) Isomorphic	W_1	L_0

However, imagine that in this same language, a syntactic input like the one in (33) is prosodified faithfully, with partial flattening. As shown in (34), this would not be predicted by the ranking of NONRECURSIVITY over sp.MATCH(XP,φ), since NONRECURSIVITY would prefer the completely flattened candidate (34b).

(33) [$_{UP}$ u [$_{WP}$ w [$_{YP}$ y z]]] → (u (w y z))

(34) NONREC >> sp.MATCH(XP,φ) favors the completely flattened candidate.

[$_{UP}$ u [$_{WP}$ w [$_{YP}$ y z]]]	NONREC	sp.MATCH(XP, φ)
☹ a. ($_1$ u ($_2$ w y z)) Partially flattened	1	1 YP
💣 b. ($_1$ u w y z) Completely flattened	L_0	W_2 WP, YP
c. ($_1$ u ($_2$ w ($_3$ y z))) Isomorphic	W_2	L_0

In this case, a subcategory-sensitive MATCH constraint could be introduced to select the correct winner. In (35), ranking sp.MATCH(XP$^{[-min]}$, φ) over NONRECURSIVITY would ensure that candidate (35a) wins. This is because both non-minimal XPs are matched in candidate (35a), while only one is in candidate (35b).

(35) sp.MATCH(XP$^{[-\mathrm{min}]}$, φ) >> NONREC favors the partially flattened candidate.

[$_{UP}$ u [$_{WP}$ w [$_{YP}$ y z]]]	sp.MATCH(XP$^{[-\mathrm{min}]}$, φ)	NONREC	sp.MATCH(XP, φ)
→ a. ($_1$ u ($_2$ w y z)) Partially flattened	0	1	1 YP
b. ($_1$ u w y z) Completely flattened	W_1 WP	L_0	W_2 WP, YP
c. ($_1$ u ($_2$ w ($_3$ y z))) Isomorphic	e_0	W_2	L_0

In this case, the Specialized constraint would predict the correct output. However, the analogous Dominance-Preserving constraint sp.MATCH(XP$^{[-\mathrm{min}]}$, φ$^{[-\mathrm{min}]}$) instead favors the isomorphic candidate (36c), because it is the only one that both matches the non-minimal XPs and preserves their relative position in the tree, as shown in (36).

(36) sp.MATCH(XP$^{[-\mathrm{min}]}$, φ$^{[-\mathrm{min}]}$) >> NONREC does not favor partial flattening.

[$_{UP}$ u [$_{WP}$ w [$_{YP}$ y z]]]	sp.MATCH(XP$^{[-\mathrm{min}]}$, φ$^{[-\mathrm{min}]}$)	NONREC	sp.MATCH(XP, φ)
☹ a. ($_1$ u ($_2$ w y z)) Partially flattened	1 WP	1	1 YP
b. ($_1$ u w y z) Completely flattened	W_2 UP, WP	L_0	W_2 WP, YP
💣 c. ($_1$ u ($_2$ w ($_3$ y z))) Isomorphic	L_0	W_2	L_0

Since the Specialized and Dominance-Preserving MATCH constraints sometimes prefer different output candidates, there will be times where only one constraint can select the right output. In the hypothetical example, the Specialized constraint was needed because it was only necessary for the non-minimal XP to be matched; it did not matter whether its relative position in the tree was preserved. There may also be cases where a Dominance-Preserving constraint is needed instead. This example shows the type of evidence one might bring to bear on the question of whether both types of constraint are needed in CON.

Future work should also investigate the different typological predictions of these constraints by comparing the factorial typologies of systems that contain sp.MATCH(XP, φ), a prosodic markedness constraint, and either a Specialized or a Dominance-Preserving constraint, and seeing which prosodic outputs are unique to each typology. These predictions should then be paired with further empirical investigation.

6.2 Eliminate subcategories altogether

Another extension of the Ban on Anti-Match Constraints would be to ban subcategory-sensitive MATCH constraints altogether, and not only those that produce Anti-Match effects. We refer to this hypothesis as the No Subcategory-Sensitive Match Hypothesis. This would mean that only the constraints in (37) are present in CON.

(37) The No Subcategory-Sensitive Match Hypothesis

a. Licit SP constraints: sp.MATCH(X^0, ω), sp.MATCH(XP, φ), sp.MATCH(CP, ι)
b. Licit PS constraints: ps.MATCH(ω, X^0), ps.MATCH(φ, XP), ps.MATCH(ι, CP)

Such an approach might be preferred by some on grounds of parsimony, since a simple ban on subcategory-sensitive MATCH constraints provides an architecturally simpler means to deal with the Anti-Match problem, while vastly simplifying the repertoire of MATCH constraints and the resulting typology.

This hypothesis receives both potential support and tentative pushback from a survey of existing analyses using subcategory-sensitive MATCH constraints. The support comes from the fact that very few analyses have used SSCs: only sp.MATCH($XP^{[+max]}$, $φ^{[+max]}$), sp.MATCH($XP^{[+max]}$, φ), and sp.MATCH($XP^{[-min]}$, φ) have been proposed, all in analyses of prosodic phrasing in Japanese (Ishihara 2014, Kalivoda 2018, Ito and Mester 2020, Bellik et al. to appear). Moreover, SSCs can sometimes be replaced by other more general constraints: for example, Ito and Mester's (2013) sp.MATCH($XP^{[-min]}$, φ) was replaced in Ito and Mester (2020) with ps.MATCH(φ, XP). We have also encountered anecdotal evidence that SSCs may sometimes be replaced. For example, an earlier version of Kalivoda and Bellik's (2021) analysis of Irish made use of sp.MATCH($XP^{[-min]}$, φ), but this was later replaced by sp.MATCH($XP_{\mathrm{OvertlyHeaded}}$, φ) (Kalivoda and Bellik, p.c.). Additionally, Van Handel's (to appear) analysis of Italian at one point included sp.MATCH($XP_{OH}^{[+max]}$, $φ^{[+max]}$) but later replaced it with ps.MATCH(φ, XP_{OH}).

However, the MATCH constraints sensitive to maximal XPs appear crucial in the analyses of Japanese (Ishihara 2014, Kalivoda 2018, Ito and Mester 2020, Bellik et al. to appear), which might suggest that we cannot entirely do away with SSCs. At the same time, Ishihara (2014) points out that sp.MATCH($XP^{[+max]}$, $φ^{[+max]}$) is conceptually similar to and performs many of the same functions as WRAP(XP). A proponent of the No Subcategory-Sensitive Match Hypothesis could try to limit the set of mapping constraints to sp.MATCH(XP, φ), ps.MATCH(φ, XP), and WRAP(XP). If some version of WRAP(XP) could replace sp.MATCH($XP^{[+max]}$, $φ^{[+max]}$) in the analyses of Japanese, then the empirical motivation for the inclusion of subcategory-sensitive MATCH constraints would disappear, since no other analyses have needed them.

Still, it is certainly plausible that subcategory-sensitive MATCH constraints could prove useful in analyses of other languages, and additional empirical support may be found once more languages have been thoroughly analyzed within Match Theory. Indeed, evidence from a larger sample of languages shows that reference to prosodic subcategories is necessary for markedness constraints, as will be seen in the following section. As a result, it would be somewhat premature to conclude that all subcategory-sensitive MATCH constraints should be eliminated from the theory based on the small sample. That is, even though SSCs may be replaced in some cases by more general constraints, and even though the only currently adopted constraints of this type could possibly be replaced by WRAP(XP), there is simply not enough typological coverage to confidently state that subcategory-sensitive MATCH constraints can always be subsumed by other constraints. However, this possibility leads us to tentatively recommend that researchers consider alternative analyses before settling on SSCs, given the potential issues they introduce.

6.3 Binary vs. privative features

One final approach to the issue of SSCs deserves mention here, though it is not necessarily an extension of the Ban on Anti-Match Constraints. Specifically, one might propose that subcategory-sensitive MATCH constraints do exist in CON, but that they may only be defined in terms of privative features, not binary features. This approach would somewhat ameliorate the proliferation of MATCH constraints, since the features [max] and [min] would have two possible values (specified or unspecified), as opposed to the three possible values for binary features (+, -, unspecified). Since there are four features (two for each argument), this means that there would be $2^4 = 16$ possible syntax-to-prosody constraints, instead of the $3^4 = 81$ constraints allowed with binary features. This number would be even smaller if specifications were limited to Specialized or Dominance-Preserving constraints.

This approach would eliminate the Anti-Match effects caused by conflicting feature values in constraints like sp.MATCH($XP^{[+max]}$, $\varphi^{[-max]}$), since privative features do not allow for conflicting values—features are either specified or unspecified. However, Anti-Match effects could still arise with privative features, because this system would still allow for constraints in which there is a feature specification on the second argument that is not also present on the first, e.g., sp.MATCH($XP^{[max]}$, $\varphi^{[min]}$), as in (38). If the constraint sp.MATCH($XP^{[max]}$, $\varphi^{[min]}$) were ranked over both sp.MATCH(XP, φ) and ps.MATCH(φ, XP), the flattened candidate (38b) would win: ignoring ZP allows the maximal YP to be mapped to a minimal φ. This means that we would still need the Ban on Anti-Match Constraints to prohibit specifications on the second argument that are not present on the first.

(38) Privative features can still cause Anti-Match effects

$[_{YP}$ y $[_{ZP}$ z]]			sp.MATCH(XP, φ)	ps.MATCH(φ, XP)	sp.MATCH(XP$^{[max]}$, φ$^{[min]}$)
→	a.	$(_1$ y $(_2$ z)) Isomorphic	0	0	1 YP
	b.	$(_1$ y z) Flattened	W_1 ZP	e_0	L_0

Independent difficulties arise with privative features. The primary issue is that there are numerous analyses that depend on the expressive power of binary features. For example, Elfner (2015) argues that LH accents in Connemara Irish appear at the left edge of non-minimal φs, and Elordieta (2015) shows that the domain of pitch reset in Lekeitio Basque is also the non-minimal φ. In order to explain why these processes apply within the non-minimal φ, it is likely that the constraints involved in these cases would need to make reference to φ$^{[-min]}$, which is something that a privative feature system for prosodic subcategories would be unable to express. Additionally, Martinez-Paricio and Kager's (2015) account of ternary rhythm using recursive feet includes ALIGN constraints that reference non-minimal feet, which would likewise not be possible in a privative feature system. Finally, the survey of previous analyses in the table in (39) shows additional markedness constraints that crucially reference [−min] or [−max] features. Though constraints (39e-k) might be definable in terms of privative features, binary features are independently needed for at least some prosodic markedness constraints.

(39) Survey of previous analyses using subcategory-sensitive markedness constraints

	Constraints	Language	Subcategory
a.	WORDMAXACCENT	Japanese (Ito and Mester 2021)	$\omega^{[+\max, -\min]}$
b.	BINMAXHEAD($\omega^{[+\max, -\min]}$)	Japanese (Ito and Mester 2021)	$\omega^{[+\max, -\min]}$
c.	EQUALSISTERS2	Japanese (Ito and Mester 2020)	Any category, [±min, ±max]
d.	Subcategory-sensitive STRONGSTART	Italian (Van Handel to appear)	Any category, [±min]
e.	BINMAX($\varphi^{[+\min]}$)	Japanese (Ito and Mester 2021)	$\varphi^{[+\min]}$
f.	NONFINALITY($\omega^{[+\max]}$)	Chukchansi Yokuts (Guekguezian 2017)	$\omega^{[+\max]}$
g.	ONSET-WORD-MAX	Non-rhotic varieties of English (Ito and Mester 2009)	$\omega^{[+\max]}$
h.	CRISPEDGE[+ATR, $\varphi^{[+\max]}$]	Akan (Kügler 2015)	$\varphi^{[+\max]}$
i.	IF-ACCENTED-THEN-HEAD-OF-$\varphi^{[+\min]}$	Tokyo Japanese, Northern Bizkaian Basque (Selkirk and Elordieta 2010)	$\varphi^{[+\min]}$
j.	IP-INITIAL-φ-BINARITY	Northern Bizkaian Basque (Selkirk and Elordieta 2010)	$\varphi^{[+\min]}$
k.	ACCENTASHEAD	Japanese (Ito and Mester 2013)	$\varphi^{[+\min]}$

To uphold the privative feature hypothesis, one would have to posit that MATCH constraints may only reference privative features, while markedness constraints may reference binary features, resulting in an unprincipled distinction between classes of constraints in CON. This does not seem well motivated, so we do not pursue this approach.

7 Conclusion

Over the past fifteen years, research on recursive prosodic structure has delivered an impressive range of empirical results. As the status of recursive prosodic subcategories has come to the fore, however, a range of theoretical questions have arisen over their interaction with the broader architecture of Match Theory. In cases where general MATCH constraints do not straightforwardly yield correct results, researchers have proposed subcategory-sensitive constraints like sp.MATCH($XP^{[+max]}$, $\varphi^{[+max]}$) (Ishihara 2014) which make direct reference to these categories. While constraints of this type may ultimately play a role in the theory, their introduction raises a number of questions about the licit shapes of MATCH constraints. Left unchecked, the possibility for unconstrained subcategory specification leads to a proliferation of logically possible MATCH constraints, including MATCH constraints that motivate non-isomorphism between syntax and prosody. In this paper, we have argued that the existence of such constraints is theoretically undesirable for two reasons: first, they undermine the impulse to simplify the mapping between syntax and prosody which has driven much of the work in Match Theory; and second, they complicate the task of subsuming MATCH constraints into the broader theory of Faithfulness. As such, we propose that they must be ruled out by the Ban on Anti-Match Constraints. Whether the extensions discussed in Section 6 are also needed remains an empirical matter to be investigated in future work.

Before drawing this paper to a close, we would like to highlight five points which have come up in the course of this investigation. The first of these points concerns the status of prosodic subcategories in the theory of prosodic structure at large. With much recent work, we take the notions of prosodic recursion and reference to subcategory specifications to reflect theoretically legitimate and empirically productive extensions of the classical Prosodic Hierarchy Theory (Ito and Mester 2009, 2013; Elfner 2015; Bennett 2018; i.a.). In light of this stance, we would like to emphasize that this chapter is not meant as an argument against the use of these subcategories in work on the syntax-prosody interface. Rather, we call for caution in extending the use of subcategories to MATCH constraints.

The second involves the status of subcategory-sensitive MATCH constraints in the literature. Although these constraints have not been invoked with great frequency, the cases which provide the most compelling evidence for their existence are those which involve the matching of maximal constituents (e.g., Ishihara 2014). This observation continues an intuition in the syntax-prosody literature, made clearly by Truckenbrodt (1999), that maximal projections appear to enjoy a special status in the mapping from syntax to prosody. We are not in a position at present to discuss the factors which might underlie this observation or the various ways in which it might be formalized, but we take it to reflect a significant pattern that requires further investigation.

The third point concerns the status of subcategory-sensitive mapping constraints. As noted above, it may be the case that it is invariably formally possible to replace SSCs with more general constraints. In light of this fact and the issues of constraint proliferation and Anti-Match effects highlighted in this chapter, we tentatively advise researchers against the use of such subcategory-sensitive MATCH constraints when alternative analyses are available. Of course, further research may show that SSCs are a necessary component of the theory, but this is less than clear at present.

The fourth point concerns the generality of the problem that we have addressed in this work. While we have focused on the effects of subcategory specifications on MATCH constraints, the issues which surface in this paper are not restricted to Match Theory: subcategory-sensitive ALIGN constraints have also been proposed (Selkirk and Elordieta 2010, Martinez-Paricio and Kager 2015, Bennett et al. 2018). While a fuller discussion is outside the scope of this chapter, we note that the use of these constraints could potentially introduce some of the same issues with the SSCs discussed here. Subcategory-sensitive ALIGN constraints present the same problem of constraint proliferation, since each constraint contains two arguments, each of which may be specified for a subcategory, as well as the directionality of alignment. Additionally, they may give rise to undesirable predictions similar to Anti-Match effects. Given that the focus of this chapter is on the use of subcategories in MATCH constraints, we leave it to future research to determine whether any pathological predictions are made by the allowance of subcategory specification at the φ level for ALIGN constraints.

The fifth and final point concerns the value of SPOT as a tool for carrying out the types of investigation which inform the theory in this way. To determine the particular feature specifications that do and do not give rise to Anti-Match effects, we considered 16 different OT Systems, each with thousands of candidates. Such work would not have been feasible by hand, but is made possible by the automatic generation of candidates in SPOT. We expect SPOT to continue to be a useful resource, both for further investigation into SSCs—such as the different predictions made by Specialized versus Dominance-Preserving constraints—and, more broadly, for comparing different approaches to the syntax-prosody interface.

References

Alderete, John D. (2001) Dominance effects as transderivational anti-faithfulness. *Phonology*, *18*(2), 201-253.

Bellik, Jennifer, Bellik, Ozan and Kalivoda, Nick (2015-2021) *Syntax-Prosody in Optimality Theory (SPOT)*. Software application. http://spot.sites.ucsc.edu

Bellik, Jennifer, Ito, Junko, Kalivoda, Nick and Mester, Armin (to appear) Matching and Alignment. In Junko Ito, Haruo Kubozono, and Armin Mester (eds.), *Prosody and Prosodic Interfaces*. Oxford University Press.

Bennett, Ryan (2012) Foot-conditioned phonotactics and prosodic constituency. PhD dissertation, University of California, Santa Cruz.

— (2018) Recursive prosodic words in Kaqchikel (Mayan). *Glossa: A journal of general linguistics*, *3*(1).

Bennett, Ryan, Harizanov, Boris, and Henderson, Robert (2018) Prosodic smothering in Macedonian and Kaqchikel. *Linguistic Inquiry, 49(2)*, 195-246.

Booij, Geert (1996) Cliticization as prosodic integration: The case of Dutch. *The Linguistic Review*, *13*, pp. 219-242.

Chen, Matthew Y. (1987) The syntax of Xiamen tone sandhi. *Phonology*, *4*, 109-149.

Chomsky, Noam and Halle, Morris (1968) The Sound Pattern of English.

Clements, George Nick (1978) Tone and syntax in Ewe. *Elements of tone, stress, and intonation*, 21-99.

Elfner, Emily (2012) Syntax-Prosody Interactions in Irish. PhD dissertation, University of Massachusetts, Amherst.

— (2015) Recursion in prosodic phrasing: evidence from Connemara Irish. *Natural Language & Linguistic Theory*, Vol. 33, No. 4, pp. 1169-1208.

Elordieta, Gorka (2015) Recursive phonological phrasing in Basque. *Phonology*, *32*(1), 49-78.

Guekguezian, Peter Ara (2017) Templates as the interaction of recursive word structure and prosodic well-formedness. *Phonology*, *34*(1), 81-120.

Ishihara, Shin (2014) Match Theory and the recursivity problem. In Shigeto Kawahara and Mika Igarashi (eds.), *Proceedings of FAJL 7: Formal Approaches to Japanese Linguistics* (pp. 69-88). Cambridge, MA: MIT Working Papers in Linguistics.

Ito, Junko and Mester, Armin (2007) Prosodic adjunction in Japanese compounds. MIT working papers in linguistics, 55, 97-111.

— (2009) The extended prosodic word. In Janet Grijzenhout and Bariş Kabak (eds.), *Phonological domains: Universals and deviations* (pp. 135-194). Berlin: Mouton de Gruyter.

— (2012) Recursive prosodic phrasing in Japanese. *Prosody matters: Essays in honor of Elisabeth Selkirk*, 280-303.

— (2013) Prosodic subcategories in Japanese. *Lingua*, Vol. 124, pp. 20-40.

— (2020) Match Theory and Prosodic Wellformedness Constraints. In Hongming Zhang and Youyong Qian (eds.) *Prosodic Studies. Challenges and Prospects*. London and New York: Routledge, 252-274.
— (2021) Recursive Prosody and the Prosodic Form of Compounds. *Languages*, *6*(2), 65.
Kager, René, and Martínez-Paricio, Violeta (2018) The internally layered foot in Dutch. *Linguistics*, *56*(1), 69-114.
Kalivoda, Nick (2018) Syntax-Prosody Mismatches in Optimality Theory. PhD Dissertation, University of California, Santa Cruz.
Kügler, Frank (2015) Phonological phrasing and ATR vowel harmony in Akan. *Phonology*, *32*(1), 177-204.
Ladd, D. Robert (1986) Intonational phrasing: the case for recursive prosodic structure. *Phonology*, *3*, 311-340.
Lee, Seunghun and Selkirk, Elisabeth (to appear) A modular theory of the relation between syntactic and phonological constituency. In Haruo Kubozono, Junko Ito, and Armin Mester (eds.), *Prosody and Prosodic Interfaces*. Oxford University Press. lingbuzz/005797
Martínez-Paricio, Violeta and Kager, René (2015) The binary-to-ternary rhythmic continuum in stress typology: Layered feet and non-intervention constraints. *Phonology, 32(3)*, 459-504.
Nespor, Marina and Vogel, Irene (1986) *Prosodic Phonology*. Dordrecht: Foris.
Peperkamp, Sharon (1997) *Prosodic words.* The Hague: Holland Academic Graphics.
Prince, Alan, Merchant, Nazarré, and Tesar, Bruce (2007-2020) *OTWorkplace.* https://sites.google.com/site/otworkplace/
Selkirk, Elisabeth (1986) On derived domains in sentence phonology. *Phonology*, Vol. 3, pp. 371-405.
— (2009) On clause and intonation phrase in Japanese: The syntactic grounding of prosodic constituent structure. *Gengo Kenkyu* 136. 35-73.
— (2011) The syntax-phonology interface. In John Goldsmith, Jason Riggle and Alan C. L. Yu (eds.) The Handbook of Phonological Theory, 2nd edition.
Selkirk, Elisabeth and Elordieta, Gorka (2010) The Role for Prosodic Markedness Constraints in Phonological Phrase Formation in Two Pitch Accent Languages. Handout of paper presented at Tone and Intonation in Europe (TIE) 4 conference, Stockholm, Sweden.
Shingler, Edward and Bellik, Jennifer. This volume. Counting tree parses. Chapter 2.
Truckenbrodt, Hubert (1999) On the relation between syntactic phrases and phonological phrases. *Linguistic Inquiry*, Vol. 30, No. 2, pp. 219-255.
Van Handel, Nicholas (to appear) Matching Overtly Headed Syntactic Phrases in Italian. *Phonology*.
— This volume. Visibility Settings for Match Theory. Chapter 6.

Vigário, Marina (1999) On the prosodic status of stressless function words in European Portuguese. In T. Alan Hall and Urula Kleinhenz (eds.) *Studies on the phonological word*, 255-294. Amsterdam: John Benjamins.
Vigário, Marina (2003) The prosodic word in European Portuguese. Berlin/New York: Mouton de Gruyter.
Wagner, Michael (2005) *Prosody and recursion*. PhD Dissertation, Massachusetts Institute of Technology.

Appendix: Syntactic inputs for all systems

# of terminal nodes	Syntactic inputs	
2-word inputs (2 total)	{y z}	{[y z]}
3-word inputs (2 total)	{w [y z]}	{[w [y z]]}
4-word inputs (6 total)	{u [w [y z]]}	{u [[w y] z]}
	{[u w] [y z]}	{[u [w [y z]]]}
	{[u [[w y] z]]}	{[[u w] [y z]]}
5-word inputs (14 total)	{s [u [w [y z]]]}	{s [u [[w y] z]]}
	{s [[u w] [y z]]}	{s [[u [w y]] z]}
	{s [[[u w] y] z]}	{[s u] [w [y z]]}
	{[s u] [[w y] z]}	{[s [u [w [y z]]]]}
	{[s [u [[w y] z]]]}	{[s [[u w] [y z]]]}
	{[s [[u [w y]] z]]}	{[s [[[u w] y] z]]}
	{[[s u] [w [y z]]]}	{[[s u] [[w y] z]]}